

05. CDC / RDC 완전 이해 — 정독용 심층 개념서

이 문서의 목적. 지원자의 가장 강력한 무기인 CDC/RDC sign-off 방법론을, 면접에서 "깊이 있게 자랑할 수 있는" 수준까지 밑바닥부터 다시 세운다. 다른 카드(`daily_study/D_cdc.md`)가 30초 암기용 압축본이라면, 이 문서는 **왜 그런가를 끝까지 따라가는 서사적 해설서**다. 외우지 말고 이해하라. 이해하면 어떤 꼬리질문도 같은 1st-principle에서 다시 유도해 답할 수 있다.

지원자 좌표. 삼성 S.LSI Design Technology에서 **CDC/RDC sign-off 방법론이 핵심 업무**였던 4년차. 실측 성과(`04_resume_selfmastery.md` 근거): RDC false violation **47,159 → 5,795건(87.7%↓)**, CDC SFR-to-HW noise **95% 자동분류·review 5%**, ML→LLM 기반 분류 PoC. 이 주제에서만큼은 면접관보다 깊을 수 있어야 한다.

메모리 브릿지 한 문장(미리). "메모리를 몰라서 걱정인 게 아니라, **메모리 Peri가 곧 내가 하던 일**입니다 — 코어/I/O/DLL·PLL/refresh가 만드는 다중 도메인, 그게 전부 CDC/RDC입니다."

0. 큰 그림 — 이 문서가 답하는 단 하나의 질문

칩 안에는 서로 박자가 안 맞는 여러 개의 시계(클럭)와 여러 개의 리셋이 동시에 돈다. 박자가 안 맞는 두 영역 사이로 신호가 넘어갈 때, **"받는 쪽이 보내는 쪽의 변화를 하필 가장 나쁜 순간에 붙잡으면 무슨 일이 벌어지는가"** — 이 한 질문이 CDC와 RDC의 전부다.

답은 **메타스터빌리티(metastability)** 다. 그리고 메타스터빌리티는 **없앨 수 없다**. 우리가 할 수 있는 건 그것이 시스템 고장으로 번질 **확률을 천문학적으로 낮추는 것(MTBF)** 뿐이다. 이 문서는 다음 사슬을 한 칸씩 끝까지 따라간다:

여러 클럭/리셋이 왜 생기나

- 비동기 경계에서 setup/hold 위반은 왜 불가피한가
 - 위반하면 FF는 왜 진동하다 임의 값이 되나 (메타스터빌리티)
 - 이게 고장으로 번지는 평균시간이 MTBF, 무엇이 좌우하나
 - 1-bit는 2-FF로 막는다 (왜 2단인가)
 - multi-bit은 왜 2-FF로 못 막나 (bit skew)
 - 그래서 gray / FIFO / handshake / MUX-hold
 - 따로 동기화한 게 다시 만나면? (reconvergence)
 - 이걸 도구로 자동 검증 (structural CDC), false violation은 왜 폭증하나
 - 클럭이 아니라 '리셋 해제 순간'이 문제면? (RDC)
 - 내가 만든 5단계 방법론 (87.7%↓)
 - 그리고 이게 전부 메모리 Peri에서 그대로 일어난다

1. 도입 — 칩에는 왜 시계가 여러 개 생기나

1.1 "클럭 도메인"이라는 말의 진짜 의미

클럭 도메인이란 같은 클럭 신호(같은 source에서 나와 같은 위상·주파수로 정렬된 클럭)로 동작하는 플립플롭(FF)들의 집합이다. 한 도메인 안에서는 모든 FF가 같은 박자에 맞춰 데이터를 주고받는다. 그래서 "이 신호가 다음 클럭 엣지 전에 안정될까?"를 정적 타이밍 분석(STA)으로 수학적으로 보장할 수 있다. 한 도메인 내부는 STA가 책임지는, 말하자면 질서가 보장된 세계다.

문제는 도메인이 둘 이상일 때다. 두 도메인의 클럭이 위상 관계가 없거나(asynchronous) 위상은 알아도 정수배가 아니면, 한쪽 FF의 출력이 변하는 순간과 다른 쪽 FF가 그 값을 붙잡는 순간 사이에 아무런 약속도 없다. 질서가 보장된 두 세계 사이에 무법지대(crossing)가 생긴다. 이 무법지대를 안전하게 건너는 기술이 CDC다.

1.2 칩에는 왜 시계가 하나가 아닌가 — 6가지 현실적 이유

신입 면접관이 "클럭이 왜 여러 개 있죠?"라고 물으면, 다음을 *이유*의 묶음으로 답할 수 있어야 한다. 단일 클럭으로 칩 전체를 돌리는 게 불가능한 이유들이다.

1. **인터페이스마다 표준 주파수가 고정돼 있다.** 외부 메모리, PCIe, USB, DDR PHY 등은 각자 규격이 정한 주파수로 동작해야 한다. 코어가 1.2 GHz로 돌든 말든 DDR I/O는 자기 규격 박자로 돌아야 한다 → 코어 도메인 ≠ I/O 도메인.
2. **전력 최적화.** 모든 블록을 최고 주파수로 돌리면 전력(dynamic power = CV^2f , f 에 비례)이 폭발한다. 한가한 블록은 느린 클럭으로, 바쁜 블록은 빠른 클럭으로 — DVFS·multi-clock이 표준이다.
3. **클럭 생성기 자체가 도메인을 만든다.** PLL은 기준 클럭을 체배(곱)해 고속 코어 클럭을 만들고, DLL은 위상을 정렬한다. PLL/DLL 출력은 입력과 위상이 다른 *새 도메인*이다.
4. **IP 재사용.** 외부에서 사온 IP(USB 컨트롤러, 코덱 등)는 자기 클럭 가정을 갖고 설계돼 있다. SoC에 통합하면 그 IP 도메인이 그대로 칩 안으로 들어온다.
5. **물리적 거리.** 큰 칩에서 한 클럭을 칩 끝까지 skew 없이 분배하는 건 점점 불가능해진다. 영역별로 클럭을 나누는 게 현실적이다.
6. **저속 관리 도메인.** 전원 관리, JTAG, 디버그, always-on 로직은 일부러 아주 느린 클럭을 쓴다(전력·면적 절약).

1.3 메모리 Peri의 다중 도메인 — "내 일이 그대로 메모리에 있다"

여기서 결정적인 브릿지가 나온다. 메모리 칩은 셀 어레이(아날로그·소자)만 있는 게 아니다. 그 바깥의 Peri(주변회로)에 거대한 디지털 RTL이 있고, 그 디지털 영역은 **본질적으로 다중 클럭 도메인**이다(qbank A5 참조):

- **코어/제어 도메인** — command/address decode, refresh controller, ECC, redundancy 제어 로직.
- **I/O 도메인** — DDR/HBM 인터페이스. 외부 규격 주파수로 동작하며 코어와 위상 무관.
- **DLL/PLL 도메인** — 외부 클럭과 내부 클럭을 정렬하기 위해 만들어지는, 정의상 새 위상의 도메인.
- **refresh / 관리 도메인** — 저속, 때로 always-on. self-refresh 진입/탈출이 도메인을 넘나든다.
- **DFI / 데이터-스트로브 경계** — 특히 DDR/HBM I/O에서 데이터(DQ)와 스트로브(DQS)가 정렬되는 지점, 그리고 그 데이터가 코어 도메인으로 captured되는 지점이 전형적인 CDC다.

그래서 "메모리 컨트롤러 코어 clock ↔ DRAM/PHY clock(DFI)", "HBM Base Die의 host-logic ↔ DRAM-stack 경계", "SFR(CSR) 설정값이 PHY/datapath clock으로 넘어가는 길" — 이 전부가 CDC다. **메모리 Peri는 CDC/RDC의 밀집 지역**이고, 그게 정확히 지원자가 4년간 한 일이다.

2. 메타스터빌리티 — 1st-principles로 끝까지

2.1 플립플롭은 왜 setup/hold 윈도우를 갖는가

FF의 심장은 **cross-coupled latch** — 두 인버터가 서로의 출력을 입력으로 물고 있는 양의 피드백(positive feedback) 구조다. 이 구조는 0과 1, 두 안정점(stable state)을 갖는다. 마치 골짜기가 둘 있는 언덕에서 공이 한쪽 골짜기에 안착하는 것과 같다.

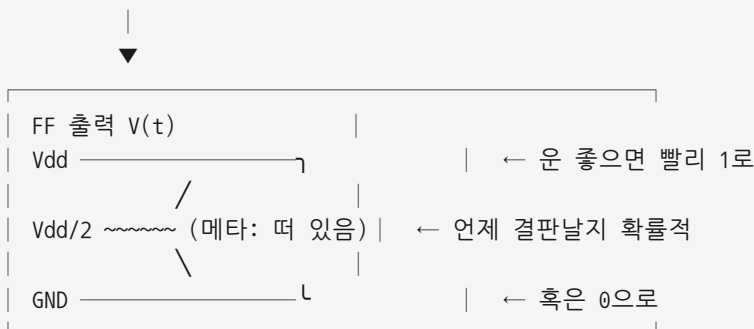
데이터를 캡처한다는 건, 클럭 엣지 순간에 이 latch를 "지금 입력값 쪽 골짜기로 밀어 넣는" 동작이다. 그런데 이 밀어 넣기가 깨끗하게 되려면 **엣지 직전 일정 시간(setup time) 동안 입력이 안정돼 있고, 엣지 직후 일정 시간(hold time) 동안에도 입력이 유지돼야** 한다. 이 두 시간을 합친 구간이 **금지된 윈도우(aperture window)** 다. 이 윈도우 안에서 입력이 **변하는** 중이면, latch는 두 골짜기 사이 언덕 꼭대기 근처로 밀려 들어간다.

2.2 위반하면 무슨 일이 — 언덕 꼭대기의 공

언덕 꼭대기(potential의 불안정 평형점)에 놓인 공은 어느 쪽으로도 굴러갈 수 있다. FF 관점에서는 출력이 **Vdd와 GND 사이의 어중간한 전압(대략 Vdd/2 근처)에 떠 있는 상태**다 — 이게 **메타스터빌리티**다. 출력은 명확한 0도 1도 아니다. 양의 피드백이 작동하기 시작하면서 출력은 한쪽으로 기울기 시작하지만, **언제 어느 쪽으로** 결판날지는 **확률적**이다.

핵심은 이 "결판(resolve)에 걸리는 시간"이 **무계(unbounded)** 라는 점이다. 공이 정확히 꼭대기에 가까울수록 굴러 떨어지는 데 더 오래 걸린다. 회로적으로는 **지수 함수**로 resolve한다 — 시간이 지날수록 메타 상태에 남아 있을 확률은 $e^{(-t/\tau)}$ 로 줄어든다. 여기서 τ 는 latch의 양의 피드백 속도를 나타내는 **resolution time constant**(현대 CMOS에서 대략 20~50 ps)다. τ 가 작을수록(피드백이 빠를수록) 빨리 결판난다.

입력이 윈도우 안에서 변함



메타 상태 잔존 확률 $\propto e^{(-t/\tau)}$

2.3 왜 비동기 도메인 간에는 위반이 불가피한가

여기가 면접의 단골 함정이자 지원자가 명료하게 짚어야 할 지점이다.

같은 도메인(동기) 안에서는 메타가 안 생긴다. 왜? 모든 FF가 같은 클럭을 쓰므로, 송신 FF의 출력이 언제 변하는지, 그게 수신 FF의 엣지 대비 어디에 도착하는지를 STA가 정적으로 계산할 수 있다. setup/hold가 지켜지도록 회로를 짜고, 안 지켜지면 STA가 위반으로 잡아 고친다. 즉 금지 윈도우를 비켜 가도록 보장할 수 있다.

비동기 도메인 사이에서는 그 보장이 원천적으로 불가능하다. 송신 클럭과 수신 클럭은 위상 관계가 없다. 송신 데이터가 변하는 순간은 수신 클럭 엣지 대비 *균등하게 아무 위치에나* 떨어질 수 있다. 두 클럭이 무한히 오래 돌면, 언젠가는 반드시 데이터 변화가 수신 FF의 금지 윈도우 안에 떨어진다. 확률 0이 아니라, **확률 1로 결국 일어난다**. 그래서 메타스터빌리티를 0으로 만드는 것은 불가능하고, 목표는 "발생해도 다음 단이 그걸 보기 전에 충분히 resolve되도록 시간을 벌어 확률을 천문학적으로 낮추는 것"이 된다.

면접 30초 골격(Q "메타가 왜 CDC에서 반드시 생기나"): "FF는 setup/hold 윈도우 안에서 입력이 변하면 출력이 Vdd/2 근처 불확정 상태로 떠 있다가 양의 피드백으로 확률적으로 결판납니다. 비동기 경계에서는 송신 데이터와 수신 클럭이 위상 무관이라 데이터 변화가 언젠가 반드시 윈도우 안에 떨어집니다 — 확률 1로요. 그래서 메타를 0으로 못 만들고 MTBF로 관리합니다."

3. MTBF — 메타가 고장으로 번지는 평균시간

3.1 "발생"과 "고장"은 다르다

메타스터빌리티가 *발생*하는 것과, 그게 *시스템 고장*으로 이어지는 것은 다른 사건이다. 메타가 발생해도, 다음 FF가 그 값을 읽기 전에 깨끗한 0이나 1로 resolve됐다면 — 비록 그 값이 옛값일지 새값일지는 몰라도 — 적어도 **메타 상태 자체는 시스템 밖으로 새어 나가지 않는다**. 시스템이 깨지는 건 "다음 단이 아직 떠 있는(메타) 값을 읽어 그게 하류 로직으로 전파될 때"다.

MTBF(Mean Time Between Failures) 는 바로 이 "메타가 결판나기 전에 하류로 새어 나가는 사건" 사이의 평균 시간이다. 잘 설계된 동기화기에서는 이게 수년~수천 년, 심지어 우주 나이를 넘기도 한다.

3.2 공식과 그 안에 담긴 직관

$$MTBF = e^{(t_r / \tau)} / (T0 \cdot f_c \cdot f_d)$$

- **t_r (resolution time, settling time)**: 메타가 결판날 때까지 **허용된 시간**. 2-FF 동기화기에서는 1단 FF가 메타가 돼도 2단이 그걸 읽기 전까지 거의 한 클럭 주기를 벌여 준다. 이 t_r이 분자에서 **지수(e^(t_r/τ))**로 들어간다 — **가장 강력한 레버**.
- **τ**: latch의 resolution time constant(피드백 속도). 작을수록 빨리 결판 → MTBF↑. 공정·소자가 결정하므로 설계자가 직접 만지긴 어렵다.
- **T0 (metastability window)**: 입력이 이 폭 안에서 변하면 메타가 발생하는 유효 윈도우. 소자 특성.
- **f_c (sampling clock)**: 수신 클럭 주파수. 높을수록 윈도우에 더 자주 노출 → MTBF↓.
- **f_d (data toggle rate)**: 입력이 토글하는 빈도. 높을수록 더 자주 위험 → MTBF↓.

3.3 무엇이 MTBF를 좌우하는가 — 면접에서 짚을 3가지

1. 가용 settling time(t_r)이 압도적으로 중요하다 — 지수 의존. FF를 한 단 더 달면(2-FF → 3-FF) t_r 이 거의 한 클럭 주기 늘어, MTBF가 수십~수만 배 좋아진다. 또는 수신 클럭을 늦춰(주기↑) t_r 을 벌 수도 있다. "FF 하나 더 = MTBF 폭발" 이 핵심 직관.
2. 주파수(f_c)가 올라가면 MTBF가 급격히 나빠진다. 고속 인터페이스일수록 같은 단수라도 MTBF가 뻥뻥해진다. → HBM/GDDR처럼 f 가 매우 높은 경계에서는 동기화기 마진을 더 봐야 한다.
3. 데이터율(f_d) 도 선형으로 작용한다. 토글이 잦은 신호 경계일수록 위험.

메모리 브릿지: HBM4가 per-pin 10 GT/s로 가면 I/O 도메인의 f_c - f_d 가 극도로 커진다. 같은 2-FF라도 MTBF 마진이 줄어든다 → 동기화기 설계와 검증의 무게가 더 커진다. "고속화는 곧 CDC 마진의 압박"이라는 한 문장으로 착지.

4. 단일 비트 동기화 — 2-FF synchronizer를 왜, 어떻게

4.1 구조

수신 도메인 클럭으로 동작하는 FF 2개를 back-to-back(둘 사이에 조합 로직 없이) 직렬로 둔다. 송신 도메인의 1-bit 신호가 첫 FF로 들어오고, 첫 FF 출력이 둘째 FF로, 둘째 FF 출력이 비로소 수신 도메인 로직으로 간다.



4.2 왜 하필 2단인가 — "첫 FF에게 결판낼 시간을 벌어 준다"

송신 신호가 수신 클럭의 금지 윈도우에 떨어지면 FF1이 메타가 될 수 있다. 여기까지는 막을 수 없다(2장의 불가피성). 핵심은 그 다음이다: FF1의 메타 출력은 다음 수신 클럭 엣지(거의 한 주기 뒤)까지 FF2가 읽지 않는다. 그 한 주기가 통째로 t_r (settling time)이 된다. 메타 잔존 확률이 $e^{(-t_r/\tau)}$ 로 줄어드는 동안, FF1은 거의 확실히 깨끗한 0이나 1로 결판난다. 그러면 FF2는 안정된 값을 캡처한다.

즉 2-FF의 본질은 "한 클럭 주기만큼의 settling 시간을 산다" 는 것이다. 그 대가로 latency가 1~2 cycle 늘어난다. 이 결과로 FF2의 출력이 메타일 확률은 천문학적으로 작아지고, MTBF는 보통 수년~수천 년이 나온다.

4.3 3단의 의미 — 그리고 왜 2단이 보통 정답인가

3-FF는 settling time을 거의 두 클럭 주기로 늘려 MTBF를 또 한 번 지수적으로 끌어올린다. 언제 3단을 쓰나? 초고주파(f_c 가 매우 커서 2단으로는 MTBF가 부족할 때), 안전등급(automotive, 의료), 또는 τ 가 나쁜 공정. 왜 항상 3단이 아닌가? latency가 또 1 cycle 늘고, 대부분의 경우 2단으로 이미 충분한(수년~수천 년) MTBF가 나오기 때문이다. 즉 "필요한 만큼만 단수를 쓴다"가 원칙.

4.4 절대 규칙 두 가지

- 두 FF 사이에 조합 로직 금지. FF1과 FF2 사이에 게이트를 넣으면 그 게이트 지연만큼 settling time을 잡아먹어 MTBF가 깨진다. 반드시 back-to-back.
- 1-bit-level 신호에만 안전하다. 다비트 버스를 비트별로 2-FF에 묶으면 — 다음 장의 재앙이 시작된다.

메모리 브릿지: PHY의 status flag(calibration done, DLL lock, PLL locked 등 1-bit level 신호)를 코어 도메인으로 올릴 때가 전형적인 2-FF 자리다.

5. 다중 비트가 위험한 이유 — bit skew

5.1 재앙의 메커니즘

다비트 버스의 각 비트는 물리적으로 다른 배선·다른 FF다. 버스 값이 바뀔 때, **비트마다 송신측에서 도착하는 시각이 미세하게 다르고(라우팅 skew), 수신측에서 메타가 결판나는 방향과 시점도 비트마다 독립적이다.** 그래서 동기화되는 그 순간, 어떤 비트는 이미 새 값으로, 어떤 비트는 아직 옛 값으로 잡힐 수 있다.

결과는 실제로는 한 번도 존재한 적 없는 **중간 조합값**이 한두 cycle 동안 수신측에 보이는 것이다.

카운터가 0111 → 1000 으로 증가 (4비트 동시 변화)
비트별 도착/resolve 시각이 제각각:

bit3: 0 → 1 (먼저 바뀜)
bit2: 1 → 0 (먼저 바뀜)
bit1: 1 → 0
bit0: 1 → 0 (늦게 바뀜)

동기화 순간 샘플 → 1111 또는 0000 같은
"있지도 않은 값"이 한 순간 보임!

주소가 잠깐 엉뚱한 곳을 가리키거나, 카운터가 잠깐 말도 안 되는 값을 갖거나, 모드 레지스터가 잠깐 미정의 모드가 된다. 이게 multi-bit을 비트별 2-FF로 묶으면 안 되는 결정적 이유 — data coherency(코히런시)가 깨진다.

5.2 그래서 "단순 2-FF 묶음 금지"

핵심은 단순하다. 2-FF는 비트 간 동시성(원자성)을 전혀 보장하지 않는다. 각 비트가 독립적으로 안전하게 건너갈 뿐, "이 비트들이 같은 시점의 일관된 값"이라는 보장은 어디에도 없다. 다비트가 *한꺼번에 의미를 갖는*(주소, 카운터, 모드값) 신호라면 반드시 코히런시를 보장하는 별도 기법이 필요하다.

면접 핵심 한 줄: "다비트를 비트별 2-FF로 묶으면 bit skew 때문에 동기화 순간 존재하지도 않는 중간 조합값이 보입니다. 그래서 gray code, async FIFO, handshake, MUX-recirculation 중 하나로 코히런시를 보장해야 합니다."

6. 해법들 — 코히런시를 보장하는 4가지 무기

다비트를 안전하게 건네는 길은 결국 두 전략 중 하나다: (A) 한 번에 1비트만 변하게 만들어 skew 자체를 무해화(gray code), 또는 (B) 1-bit 제어 신호로 다비트 데이터의 "유효 시점"을 통제(FIFO 포인터·handshake·MUX·hold). 네 가지를 본질부터 본다.

6.1 Gray code — "한 번에 1비트만 변한다"

Gray code는 인접한 값끼리 정확히 1비트만 다르도록 만든 이진 인코딩이다(예: 000→001→011→010→110...). 이게 CDC에 강력한 이유: 값이 한 단계씩 변할 때 **변하는 비트가 단 하나**이므로, skew가 있어도 수신측은 항상 "직전 값" 아니면 "현재 값" 만 본다. 절대 invalid한 중간 조합이 안 생긴다. 메타 위험 비트도 한 개뿐이라, 그 비트가 옛값/새값 어느 쪽으로 결판나든 둘 다 유효한 값이다.

한계: 인접 값 사이에서만 성립한다. 연속 증가(카운터)에는 완벽하지만, 임의로 점프하는 다비트 값(여러 비트가 동시에 바뀌는 일반 데이터)에는 못 쓴다. 그래서 gray code의 대표적 용도는 포인터(연속 증가/감소하는 인덱스)다.

6.2 Async FIFO — gray 포인터의 결정판

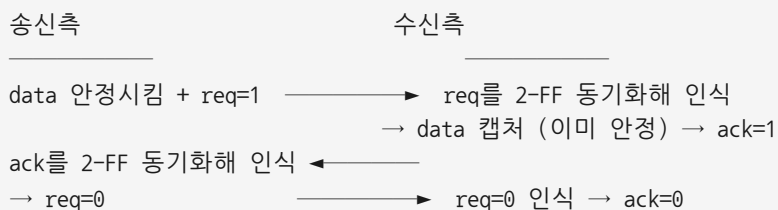
서로 다른 두 클럭 사이에 다비트 데이터 스트림을 흘려보내는 표준 구조.

- **구성:** 양쪽 클럭이 접근하는 dual-port RAM + 쓰기 도메인의 `wr_ptr` + 읽기 도메인의 `rd_ptr`.
- **포인터만 동기화한다:** 각 포인터를 gray로 변환 → 반대 도메인에 2-FF 동기화 → 비교해서 full/empty를 만든다. 6.1의 gray 덕분에 포인터가 안전하게 건너간다.
- **데이터 자체는 동기화 안 한다:** 데이터는 RAM에 그냥 쓰여 있고, 포인터가 "여긴 써도/읽어도 된다"를 보장하는 순간에만 접근하므로 안전하다.
- **깊이 2^n 이면 포인터는 $n+1$ bit:** 추가된 MSB(wrap 비트)로 full(하위 n 비트 같고 MSB 다름)과 empty(완전히 같음)를 구분한다.
- **보수적 판정(중요한 직관):** 동기화 latency 때문에 상대 포인터가 항상 "약간 과거값"으로 보인다. 그래서 실제보다 더 full/더 empty로 판단한다 → 절대 overflow/underflow가 안 나는 안전한 pessimism. "틀려도 안전한 쪽으로 틀린다"가 핵심.

메모리 브릿지: 코어 clock ↔ 메모리 clock 사이 데이터 버퍼링(rate matching)의 표준이 async FIFO다.

6.3 Handshake (req/ack) — 느리지만 가장 robust

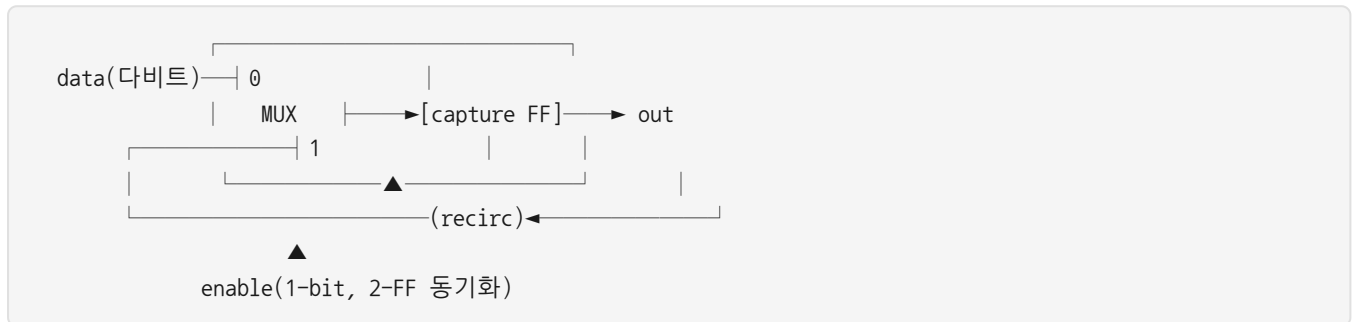
throughput이 필요 없고 가끔 다비트 control을 확실히 넘길 때 쓴다. 4-phase 흐름:



핵심: **data** 버스는 req가 안정적으로 인식될 때까지 변하지 않으므로 동기화 불필요(false path). req라는 1-bit 신호의 동기화가 "지금 data가 안정됐다"는 시점을 통제한다. 단점은 왕복(round-trip) 동기화 지연으로 느리다. (2-phase는 레벨 toggle로 더 빠르지만 상태 추적 필요, 4-phase는 매번 0 복귀로 단순·robust.)

6.4 MUX recirculation / data-hold + enable 동기화

다비트 데이터는 동기화하지 않고 경계 MUX의 한 입력에 그대로 둔다. **enable(load)** 1-bit만 2-FF 동기화한다. enable이 떨어진 동안 캡처 FF는 자기 값을 되먹임(recirculation)해 유지하고, 동기화된 enable이 떴을 때만 새 데이터를 load한다. 데이터는 enable이 인식되는 순간 이미 안정돼 있으므로 안전하고 false-path 처리한다.



전제: **enable**이 송신측에서 데이터보다 충분히 늦게 뜨도록 설계(데이터 먼저 안정 → 그다음 enable). 정적 검증에서 이 가정을 assertion으로 확인한다. **shadow register(double buffering)** 도 같은 계열 — 본 레지스터 옆에 그림자 레지스터를 두고 valid 펄스 동기화 시점에 한꺼번에 복사해 atomic update를 보장(면적 2배).

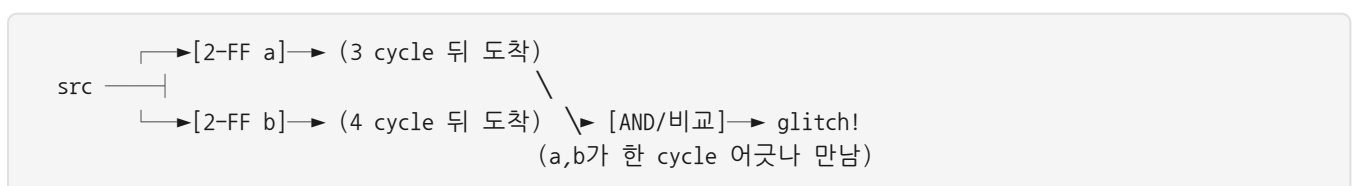
6.5 보너스 — 빠른→느린 펄스 유실과 toggle 동기화기

빠른 도메인의 1-cycle 펄스가 느린 도메인 클럭 주기보다 좁으면 수신측이 한 번도 sampling 못 해 통째로 놓친다. 해법은 **toggle 동기화기**: 송신측에서 펄스마다 레벨을 toggle → 그 레벨을 2-FF 동기화 → 수신측에서 edge detect(XOR)로 펄스 복원. 레벨은 유실 안 되므로 안전하다. 단, 다음 toggle까지 수신 2~3 cycle 간격을 보장해야 한다(연속 펄스는 사이에 FIFO/handshake로 backpressure).

7. Reconvergence — 따로 건넌 신호가 다시 만나면

7.1 문제

한 source에서 갈라진 여러 신호를 각각 따로 동기화한 뒤, 수신 도메인에서 다시 한 로직으로 합치면 (reconverge), 비트마다 동기화 latency가 달라 **도착 cycle이 어긋난다**. 그 어긋남이 합류 지점에서 glitch나 논리적 불일치를 만든다. 5장의 bit skew가 "비트별 2-FF"의 문제였다면, reconvergence는 "정상적으로 동기화한 신호들조차도 재합류하면 위험"하다는, 한 단계 더 미묘한 함정이다.



7.2 회피 원칙

- 한 번만 동기화하고 fan-out 하라. 갈라지기 전에 한 번 동기화한 뒤 수신 도메인에서 fan-out하면 어긋남이 없다.
- 다비트로 함께 의미를 가지면 묶어서 건네라. gray code / async FIFO / handshake / MUX-hold로 코히런시를 보장한 채 한꺼번에 건넨다 — 즉 6장의 기법이 reconvergence의 해법이기도 하다.
- quasi-static / false-path는 명시적으로 관리하라. 동작 중 거의 안 변하는 config는 static처럼 다루되, 바뀌는 순간엔 여전히 "sampled when stable"을 보장해야 한다. false path(STA에서 타이밍 안 잡는 경로)는 진짜 동기화기 뒤에 있는지 구조 검증 후에만 set_false_path로 제외한다 — 잘못 걸면 진짜 위반을 놓친다.

8. 정적(structural) CDC 검증 — 그리고 false violation은 왜 폭증하나

8.1 CDC sign-off의 4축

상용 도구(SpyGlass CDC, Questa CDC, VC SpyGlass 등)가 하는 일을 네 축으로 정리한다:

1. 구조(structural) 검증: 설계의 모든 clock crossing을 자동 탐지하고, 각 경로에 적절한 동기화 구조(2-FF / FIFO / handshake)가 있는지, 다비트는 gray/MUX 구조인지 확인한다.
2. 기능(functional) 검증: 동기화 프로토콜·data-stability를 SVA assertion으로 생성해 시뮬/formal로 검증한다 (예: "enable이 인식되는 동안 data는 안 변한다").
3. RDC: reset domain까지 본다(9장).
4. Waiver(면제): 안전하다고 입증된 crossing(false path, quasi-static)에 근거(reason code)를 달아 violation을 정식 제외하고, 회귀 방지로 버전 관리한다.

8.2 false violation은 왜 폭증하는가 — 지원자의 진짜 전문성

이게 면접에서 깊이를 보여줄 핵심 지점이다. 구조는 맞는데 도구가 모르는 의도가 false violation의 근원이다.

도구는 보수적이다. "이 crossing이 안전하다는 증거를 내가 못 찾으면 일단 위반으로 보고한다"가 기본 자세다. 그런데 안전성의 상당 부분은 설계자의 머릿속에만 있는 의도다:

- 이 다비트는 사실 quasi-static이다 — logic이 disable일 때만 쓰이고 이후 불변. 도구는 SW가 그렇게 쓴다는 약속을 모른다.
- 이 두 신호는 절대 동시에 active되지 않는다 — 모드가 상호배타적이다. 도구는 모른다.
- 이 경로는 의도된 false path다 — 데이터 버스는 enable이 통제하므로 타이밍 안 잡아도 된다. 도구는 모른다.

도구는 모든 가능한 시나리오의 조합을 가정하므로, 실제로는 동시에 일어나지 않는 조합의 위반까지 전부 쏟아낸다 → 수만 건의 noise. 진짜 위험(실제로 동기화기가 없는 crossing)은 그 noise에 파묻힌다.

8.3 constraint로 noise를 줄인다 — 그리고 false negative의 위험

해법은 설계 의도를 constraint로 도구에 전달하는 것이다: 어떤 clock이 비동기인지(set_clock_groups), 어떤 경로가 false path인지, 어떤 신호가 quasi-static인지를 명시한다. 그러면 도구가 그 의도를 반영해 noise를 걷어낸다.

하지만 여기에 가장 위험한 함정이 있다 — **false negative**(진짜 위반을 놓침). constraint나 waiver를 남발하면 진짜 버그를 숨긴다. 그래서 지원자의 일관된 원칙은:

- waiver는 **reason-coded·리뷰·버전관리**한다.
- 자동화로 "**신규 vs 기존 waiver**" 를 구분해 회귀(이전에 없던 새 위반이 waiver에 묻히는 것)를 차단한다.
- "**확실히 안전한 것만 깎고, 애매하면 위반으로 남겨 사람이 본다**" — 보수적 원칙.

지원자 서사(CDC SFR-to-HW 95% 분류): "APB write 트랜잭션 자체는 handshake로 동기화되지만, SFR 출력이 logic 도메인으로 가는 config path는 그 보호 밖이라 각각이 CDC가 됩니다. 대부분 quasi-static인데 일부 command성 register가 섞여 false가 폭증하죠. 저는 register의 *의도*(configuration인지 command인지, quasi-static인지)를 IP-XACT spec으로 표준화해 CDC constraint를 자동생성하고, spec 작성 부담은 LLM 추론으로 덜어 SFR-to-HW path noise의 95%를 자동 분류하고 review를 5%로 줄였습니다. 핵심 통찰은 '문제가 분류 알고리즘이 아니라, 설계 의도가 EDA 도구까지 전달되지 않는 것'이었습니다."

9. RDC — Reset Domain Crossing, 클럭이 아니라 **리셋 해제 순간**이 문제

9.1 RDC란 무엇인가 — CDC의 reset 버전

RDC는 서로 다른 리셋 도메인에 속한 두 FF 사이의 신호 전달 문제다. 송신 FF는 리셋 A로, 수신 FF는 리셋 B로 리셋된다고 하자. CDC가 "다른 클럭"이 만드는 위험이라면, RDC는 "다른 리셋"이 만드는 **같은 종류의** 위험이다 — 둘 다 비동기 변화가 캡처 엣지 근처에 도착해 메타가 난다.

9.2 결정적 차이 — 엣지가 아니라 "**de-assert 타이밍**"

여기가 RDC를 CDC와 구별하는 가장 중요한 포인트다.

리셋은 보통 **비동기로 assert**된다(클럭과 무관하게 즉시 0으로 만든다). assert 자체는 안전하다 — 그냥 출력을 0으로 누르는 거니까. 문제는 **de-assert(리셋 해제) 순간**이다.

리셋 A가 떨어지는(de-assert) 순간, 송신 FF의 출력이 **클럭과 무관하게** 리셋 값에서 정상 동작 값으로 변한다. 이 변화가 하필 수신 FF(리셋 B는 아직 안 풀렸거나 다른 타이밍)의 **클럭 엣지 근처(recovery/removal 윈도우)에 도착하면** — 수신 FF가 메타가 되거나 glitch가 발생한다.

용어 정리(STA 연계): 리셋도 클럭 엣지 대비 de-assert 타이밍을 지켜야 한다. **removal time**(리셋 해제가 엣지 후 일정 시간 유지 — hold와 유사)과 **recovery time**(리셋 해제가 엣지 전 일정 시간 안정 — setup과 유사). 이걸 어기면 메타. RDC는 본질적으로 "리셋 de-assertion이 만드는 recovery/removal 위반"이다.

CDC: 데이터 변화가 수신 *클럭 엣지*의 setup/hold 윈도우에 떨어짐
RDC: *리셋 de-assert*로 인한 송신 FF 변화가
수신 FF 클럭 엣지의 recovery/removal 윈도우에 떨어짐
→ 같은 메타, 다른 트리거(클럭이 아니라 리셋 해제 순간)

9.3 reset tree / dependency 분석

리셋은 보통 단순하지 않다. power-on reset, soft reset, JTAG reset, mode-change reset 등 여러 소스가 트리(reset tree)를 이루고, 서로 의존한다(예: soft reset이 hard reset에 종속 — hard가 풀려야 soft가 의미를 가짐). RDC 분석의 핵심은 이 **reset source 식별 + 의존성(dependency) 분석**이다: 어떤 리셋들이 항상 함께 동작하고, 어떤 리셋들이 독립적으로 풀릴 수 있는가.

9.4 false violation이 왜 폭증하나 — RDC 버전

CDC와 같은 병이지만 더 심하다. 도구(예: Meridian RDC)는 기본적으로 **모든 리셋 시나리오의 조합**을 가정한다. 실제로는 동시에 일어나지 않는 리셋 조합, 또는 항상 함께 풀려 위험이 없는 조합의 위반까지 전부 보고한다. 그래서 지원자의 실제 사례에서 **47,159건**이라는 false가 쏟아졌다. 리셋이 N개면 조합은 폭발적으로 늘어나기 때문이다.

핵심은 CDC와 동일: "**어떤 리셋이 함께 동작하는지**"라는 의존성(의도)을 도구에 전달하면 noise가 사라진다.

10. 지원자 방법론 서사 — 면접에서 설명하는 버전

이 장은 지원자의 실측 성과를 "면접관 앞에서 자랑스럽게, 그러나 정확하게" 푸는 대본이다.

10.1 RDC Reset Dependency Analysis — false violation 87.7%↓ (47,159 → 5,795)

한 줄 핵심(부분집합 판단): *aggressor(송신 FF)의 리셋 소스 집합이 victim(수신 FF)의 리셋 소스 집합의 부분집합이면, RDC 위반이 불가능하다.*

왜 안전한가: aggressor를 리셋시키는 모든 조건이 victim도 리셋시킨다 → **aggressor가 리셋될 때 victim도 항상 함께 리셋된다** → victim이 "동작 중에 변하는 값을 샘플링하는" 상황 자체가 없다 → safe → rdc_false_path 로 처리.

5단계 절차:

- ① reset source 식별 — 문서 + RTL에서 모든 리셋 소스 추출
- ② 의존성 시뮬레이션 — 어떤 리셋이 항상 함께 켜지는지 실제로 확인
- ③ 소스 간 dependency Matrix — 리셋 쌍별 의존/독립 관계를 매트릭스로
- ④ safe 쌍 → TCL 자동생성 — 부분집합 성립(safe=0) 쌍을 RDC Tool용 constraint로
- ⑤ 설계 변경 시 Matrix 자동 갱신 — 회귀 방지

면접 30초 스크립트: "RDC는 서로 다른 리셋 도메인 간 신호가 비동기 리셋 de-assert 타이밍 때문에 메타가 나는 문제입니다. 툴이 모든 리셋 조합을 가정해 4.7만 건 false가 났는데, 저는 '송신 리셋 소스가 수신 리셋 소스의 부분집합이면 둘은 항상 같이 리셋되어 위반이 불가능하다'는 논리로 safe path를 자동 판정하는 5단계 매트릭스 방법론을 만들어 87.7% 줄였습니다."

꼬리질문 방어: - "부분집합이면 정말 안전한가, 반례는?" → 전제는 "aggressor 리셋 $\subseteq$ victim 리셋"이 **모든 동작 모드에서 성립**한다는 것. 그래서 ②의 시뮬레이션으로 리셋 간 실제 의존을 확인해 매트릭스를 만든다. 전제가 깨지는 모드가 있으면 그 쌍은 safe로 넣지 않는다. - "*false negative(진짜 위반을 놓침)*는 어떻게 막았나?" → **가장 중요한 리스크**. 매트릭스를 보수적으로 만든다 — 시뮬레이션에서 독립적으로 de-assert 가능한 리셋 쌍은

절대 safe로 두지 않고, 불확실하면 위반으로 남겨 사람이 본다. "확실히 안전한 것만 깎고 애매하면 남긴다." - "왜 simulation으로? formal은?" → 리셋 소스의 논리적 의존(soft reset이 hard reset에 종속 등)을 빠르게 전수 확인하려 시뮬레이션을 썼다. formal/assertion으로 매트릭스를 교차검증하면 더 견고하다(개선점). - "RDC와 CDC 차이 한 줄?" → CDC=다른 클럭, RDC=다른 리셋. 둘 다 비동기 변화가 캡처 엣지 근처에 와서 메타가 나는 문제.

10.2 CDC SFR-to-HW — noise 95% 자동분류, review 5%

(8.3에서 다룬 서사의 방법론 골격을 다시 정리)

문제의 본질: APB/AHB write 트랜잭션은 handshake로 동기화되지만, 그 SFR 출력이 logic 클럭 도메인으로 직접 fan-out되는 **configuration path**(clock divider 값, mask, mode 등)는 handshake 보호 밖이라 각각이 CDC다. multi-bit이 동시에 바뀌면 bit skew.

의도별 분기(반드시 구분): - **quasi-static / false path** — logic disable 중에만 쓰이고 이후 불변. 동기화 회로 불필요, 타이밍 false path. 단, SW 약속이 깨지면 버그 → assertion으로 "enable 중엔 안 바뀐다"를 검증. (면적·전력 0이 장점) - **shadow register(double buffering)** — multi-bit atomic update. 면적 2배. - **MUX-enable(recirculation)** — 데이터 직접 연결, enable/load만 2-FF 동기화. 데이터가 enable 도착 전 stable해야 (open-loop). - **2-FF** — 1-bit 전용. multi-bit엔 부적합.

지원자의 접근(핵심 통찰): "path마다 의도가 다른 게 본질 문제"라고 재정의 → RTL flow에 **CSR spec 표준화 단계(IP-XACT 기반)** 를 끼워 register별 의도를 spec으로 전달 → 거기서 CDC constraint/assertion을 자동생성. spec 작성 부담은 (1) legacy register 속성을 RTL+SW 사용패턴으로 **LLM 추론**, (2) spec field 초안 보조, (3) CDC report를 register 단위로 **클러스터링**으로 덜었다.

왜 ML 단독 분류는 실패했나(2년 전 PoC의 교훈): CDC는 static 검증이 100% 정확해야 sign-off로 넘어가는데, ML 분류의 잔여 오류를 신뢰할 근거가 없었다. 진짜 원인은 "분류 알고리즘"이 아니라 "**설계 의도가 EDA 도구까지 전달되지 않는다**" 였다. 그래서 spec-driven으로 의도를 전달하는 쪽으로 재정의했다. — 이 "**재정의**" 서사가 면접에서 사고의 깊이를 보여주는 최강 카드다. ("AI는 sign-off 대체가 아니라, 설계자가 표준화 flow에 들어오게 돕는 보조다"라는 일관된 철학으로 착지.)

10.3 한 단계 위 — RTL Sign-off Agent (false 20%↓, 사업부 1호)

CDC/RDC를 넘어 Lint/CDC sign-off 전반으로 확장한 그림. rule 특성에 따라 분석을 분기: **구조적으로 명확한 rule = Tool DB 기반 deterministic logic**, 설계 의도가 들어가는 rule = RAG로 도메인 지식 주입한 LLM 또는 LLM 단독 추론. 근거 출처별 신뢰도 스코어링으로 "왜 그렇게 판단했는가"를 사람이 검증 가능하게 남긴다. 핵심 철학은 여기서도 동일: **sign-off 대체가 아니라 우선순위·출발점 제시, 최종 판단은 사람.**

11. 메모리 연결 — "내가 하던 CDC/RDC가 그대로 핵심"

이제 다리를 완전히 건넌다. 면접의 본질은 "삼성 로직 출신이 메모리에서 뭘 하느냐"이고, 답은 "**Peri의 디지털 영역은 CDC/RDC 밀집 지역이고, 그게 정확히 내 4년**" 이다.

11.1 Peri의 다중 도메인이 곧 CDC 밀집 지역

- **코어 ↔ I/O 도메인** — 메모리 컨트롤러 코어 clock과 DRAM/PHY clock(DFI 경계)은 위상 무관. command/address/data가 이 경계를 넘는다. 다비트 command(주소, burst length)는 bit skew 방지가 필수 → gray/FIFO/handshake/MUX-hold.
- **DLL/PLL 도메인** — 외부 클럭과 내부 클럭을 정렬하느라 만들어지는 새 위상의 도메인. lock/done 같은 1-bit status는 2-FF로 코어에 올린다.
- **refresh / self-refresh 관리 도메인** — 저속·always-on. self-refresh 진입/탈출 이벤트가 도메인을 넘는다(빠른 → 느린 펄스 → toggle 동기화기 자리).
- **SFR → PHY-clock config path** — PHY의 **training/calibration 파라미터**를 register로 설정하는 길이 정확히 SFR-to-HW CDC다. 다비트 config는 update 펄스 동기화 후 한꺼번에 load. "설정 path도 CDC"라고 말하면 신입이라도 강한 인상.

11.2 DDR/HBM I/O — 데이터-스트로브 도메인

DDR/HBM I/O에서는 **데이터(DQ)**와 **스트로브(DQS)**의 정렬, 그리고 그 데이터가 코어 도메인으로 captured되는 지점이 전형적 CDC다. 특히 HBM4가 **per-pin 10 GT/s, 2048-bit I/O**로 가면 f_c·f_d가 폭증해 같은 단수라도 MTBF 마진이 뻥뻥해진다 → 동기화기 설계와 검증의 무게가 커진다. "**고속·광폭화 = CDC/SI/타이밍 마진의 압박 = 내 sign-off 자동화의 가치↑.**"

11.3 RDC도 그대로 이식 가능

HBM Base Die·메모리 컨트롤러도 power-on reset / soft reset / mode-change reset / JTAG reset이 얹혀 RDC noise가 크다. 10.1의 dependency matrix 방법론을 Peri/컨트롤러에 **그대로 적용** 가능하다.

착지 문장(외워 둘 것): "메모리를 몰라서가 아니라 Peri가 곧 제가 하던 일입니다.

코어/I/O/DLL·PLL/refresh가 만드는 다중 도메인, SFR이 PHY로 넘어가는 config path, 다중 reset 도메인 — 전부 제가 4년간 sign-off하고 자동화한 CDC/RDC입니다. 메모리 디바이스 디테일은 빠르게 채우겠지만, **디지털 검증 문제는 로직이든 메모리든 같습니다.**"

12. 면접 연결 — 6대 질문 30초/1분 골격

각 질문에 대해 **30초(핵심)**와 **1분(꼬리 대비 확장)** 골격을 둔다. 외우지 말고 위 장(章)의 1st-principle에서 즉석 재구성하는 연습을 하라.

Q1. "CDC가 무엇인가요?"

- **30초:** 비동기/위상차 클럭 도메인 간 신호 전달 시 수신 FF의 setup/hold 위반으로 메타스터빌리티가 생기는 문제, 그리고 그걸 안전하게 건네는 기법(2-FF, FIFO, handshake)의 총칭입니다.
- **1분 확장:** 한 도메인 안은 STA로 타이밍을 보장하지만, 비동기 경계는 위상 관계가 없어 데이터 변화가 언젠가 반드시 금지 윈도우에 떨어집니다 → 메타 불가피 → 0으로 못 만들고 MTBF로 관리 → 1-bit는 2-FF, multi-bit은 bit skew 때문에 gray/FIFO/handshake/MUX-hold. (→ 메모리 Peri 다중 도메인으로 브릿지)

Q2. "메타스터빌리티란?"

- **30초:** FF의 cross-coupled latch가 금지 윈도우 안에서 입력이 변하면 두 안정점 사이 불안정 평형($V_{dd}/2$ 근처)에 떠 있다가, 양의 피드백으로 확률적으로 0/1로 resolve하는 현상입니다. resolve 시간은 무계라 $e^{(-t/\tau)}$ 로 줄어듭니다.
- **1분 확장:** 언덕 꼭대기의 공 비유 → resolve가 무계인 게 핵심 → 그래서 다음 단이 읽기 전에 settling time을 벌여 확률을 낮추는 게 2-FF의 본질. (한 도메인 내부는 왜 안 생기나도 같이.)

Q3. "2-FF synchronizer는 왜 2개인가요?"

- **30초:** 1단 FF가 메타가 돼도, 2단이 그걸 읽기 전까지 거의 한 클럭 주기를 settling time으로 벌여 줘 거의 확실히 resolve되게 합니다. 그래서 2단 출력이 메타일 확률이 지수적으로 작아집니다.
- **1분 확장:** $MTBF = e^{(t_r/\tau)/(T_0 \cdot f_c \cdot f_d)}$ 에서 t_r 이 지수 → FF 한 단 = MTBF 폭발. 3단은 더 안전하나 latency·보통 2단으로 충분. 두 FF 사이 조합 로직 금지, 1-bit-level 전용.

Q4. "다중비트는 왜 2-FF로 안 되나요?"

- **30초:** 비트마다 도착·resolve 시각이 달라 동기화 순간 일부는 새 값, 일부는 옛 값으로 잡혀 존재하지도 않는 중간 조합값이 보입니다(bit skew). 0111→1000에서 0000이나 1111이 순간적으로요.
- **1분 확장:** 그래서 코히런스 보장 기법 — gray(한 번에 1비트, 포인터용), async FIFO(gray 포인터 동기화), handshake(req/ack로 data 유효 시점 통제), MUX-recirculation(enable만 동기화). (메모리: 주소·burst length를 PHY로 넘길 때.)

Q5. "RDC vs CDC?"

- **30초:** CDC는 다른 클럭, RDC는 다른 리셋이 만드는 같은 종류의 메타입니다. 결정적 차이는 트리거 — RDC는 클럭 엣지가 아니라 비동기 리셋 de-assertion 순간의 변화가 수신 FF의 recovery/removal 윈도우를 때립니다.
- **1분 확장:** reset tree-dependency 분석이 핵심 → 툴이 모든 리셋 조합을 가정해 false 폭증 → "aggressor 리셋 $\subseteq$ victim 리셋이면 항상 함께 리셋되어 안전"이라는 부분집합 논리로 5단계 매트릭스 방법론 → 87.7%↓. (메모리 다중 reset에 이식.)

Q6. "CDC false violation의 흔한 원인은?"

- **30초:** 구조는 맞는데 도구가 설계 의도를 모르는 경우입니다 — quasi-static인데 모르거나, 상호배타적 모드인데 동시 가정하거나, 의도된 false path인데 모르거나. 도구는 모든 시나리오 조합을 보수적으로 가정하니 noise가 폭증합니다.
- **1분 확장:** 해법은 의도를 constraint로 전달 → 단 false negative(진짜 위반 숨김)가 최대 위험 → reason-coded waiver, 신규/기존 구분 자동화, "확실한 것만 깎고 애매하면 남긴다." (→ 내 SFR-to-HW 95% 분류 서사.)

13. 스스로 점검 (10문항) — 막힘 없이 구술되는지 확인

각 항목을 보지 않고 소리 내어 답해 보고, 막히면 해당 장으로 돌아가라.

1. 칩에 클럭이 여러 개 생기는 이유를 6가지 중 4가지 이상 말할 수 있는가? 그리고 메모리 Peri의 4개 도메인을 즉시 나열할 수 있는가? (1장)

2. "한 도메인 안에서는 왜 메타가 안 생기는데 비동기 경계에서는 확률 1로 생기는가"를 STA 보장 가능/불가능 관점으로 설명할 수 있는가? (2장)
3. MTBF 공식을 적고, **t_r이 왜 지수로 들어가는 게 결정적인지**, f_c가 커지면 왜 나빠지는지 말할 수 있는가? (3장)
4. "2-FF가 settling time을 산다"는 표현을 한 클럭 주기 settling으로 풀어 설명하고, 두 FF 사이 조합 로직 금지 이유를 댈 수 있는가? (4장)
5. 0111→1000 예시로 bit skew가 만드는 "있지도 않은 중간값"을 그림 없이 말로 설명할 수 있는가? (5장)
6. gray code가 왜 **포인터에만** 강하고 임의 다비트엔 못 쓰는지, async FIFO가 왜 "보수적으로 틀려서 안전한지"를 설명할 수 있는가? (6장)
7. reconvergence가 "정상 동기화한 신호들조차 위험한" 이유와, "한 번만 동기화 후 fan-out" 원칙을 댈 수 있는가? (7장)
8. CDC false violation이 폭증하는 근본 원인("구조는 맞는데 도구가 모르는 의도")과 false negative의 위험, waiver 관리 원칙을 한 호흡에 말할 수 있는가? (8장)
9. RDC가 CDC와 다른 **결정적** 차이(클럭 엣지가 아니라 리셋 de-assertion, recovery/removal 윈도우)를 한 문장으로 못 박을 수 있는가? (9장)
10. **부분집합 논리(aggressor 리셋 ⊆ victim 리셋 → 안전)** 와 5단계 절차, 그리고 "false negative를 막으려 보수적으로 매트릭스를 만든다"는 방어 논리를 47,159→5,795(87.7%↓) 수치와 함께 자랑스럽게 설명할 수 있는가? (10장)

연계 문서: 압축 암기본 [daily_study/D_cdc.md](#) · 지원서 방어 [04_resume_selfmastery.md](#) (RDC #1, CDC SFR #2, Sign-off Agent #3) · 질문은행 [03_interview_job_qbank.md](#) (C1 CDC, C2 RDC, C9 DLL/PLL) · STA 연계(recovery/removal) [daily_study/E_sta](#) .

핵심 한 문장으로 끝: "메타스터빌리티는 못 없애고 MTBF로 관리한다. 비동기 경계의 안전성은 결국 **설계 의도를 도구에 전달하는 문제**이고, 그게 CDC든 RDC든 로직이든 메모리든 같다 — 그게 내가 4년간 한 일이다."